

SECJ3483 - WEB TECHNOLOGY

GROUP LAB ASSIGNMENT

Security Investigation and Fix

Prepared by:

Name	Matric No.
Benjamin Chew Jun Jie	A23CS0210
Tan Zhen Li	A23CS5025
Tan Zheng Yu	A23CS5017
Teow Zi Xian	A23CS0279

1. Investigation Table

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
1	Add BMI with invalid data	Backend should reject invalid data	The API returned 201 Created and created a BMI record even though the data was invalid. It accepted an empty name, age -5, height 0, and weight -70.	Yes	The backend does not validate the BMI input before saving it. It trusts the request body/frontend data and directly inserts the values into the database. This allows invalid health records to be stored. The backend should reject invalid values and calculate BMI/category itself.
2	Access another user's BMI record	Backend should deny access	The API returned 200 OK and displayed another user's BMI record when using a token from user 1.	Yes	The backend does not check whether the requested BMI record belongs to the logged-in user. It only uses the record ID from the URL and returns the data directly. This causes broken object level authorization because users can access other users' records by changing the ID.
3	Access staff route as normal user	Backend should deny access	The API returned 200 OK and displayed all BMI/person records even though the request was made by a normal user.	Yes	The backend did not check whether the logged-in user has the staff/admin role before allowing access to <code>/api/staff/persons</code> . This causes unauthorized access / broken access control because a normal user can view other users' BMI records. The route should only return data when the JWT user role is staff/admin.
4	Access admin route as normal user	Backend should deny access	The API returned 200 OK and exposed all user records, including sensitive fields such as passwords and password hashes, even when accessed by a normal user.	Yes	The backend does not verify whether the requester has the admin role before allowing access to <code>/api/admin/users</code> . This results in Broken Access Control and Sensitive Data Exposure , allowing unauthorized users to view confidential user information. The endpoint should only be accessible to administrators and should return 403 Forbidden for non-admin users. Passwords and password hashes should never be included in API responses.
5	Submit XSS payload in notes	Payload should display as text	The backend accepted and stored the XSS payload in the notes field. Postman returned it	Yes	The API storing the payload does not by itself prove an XSS vulnerability. XSS occurs only if the Vue frontend later renders

			as a JSON string, and no alert was executed.		the stored notes as HTML, such as with v-html or innerHTML. You must open the BMI record in the frontend to complete the test.
6	Check API response	Password/hash should not be visible	The API returned user details together with sensitive fields such as password: password123 and password_hash: password123 in the response from /api/profile.	Yes	The backend exposes sensitive user data in the API response. Passwords and password hashes should never be sent to the client. This is a sensitive data exposure vulnerability. The backend should return only required fields such as id, name, email, and role.
7	Try SQL Injection input	Input should be treated as data	The login request using aiman@student.utm.my' -- with password anything returned 200 OK and logged in successfully, even though the password was wrong.	Yes	The backend directly joins user input into the SQL query. The ' -- payload comments out the password check, allowing login without the correct password. This is an SQL Injection vulnerability. The backend should use prepared statements and parameter binding.
8	Try to update protected fields	Backend should reject or ignore protected fields	The backend accepted and updated the protected fields user_id, bmi, and category. The record owner was changed to user ID 2, while the BMI was changed to 10 and the category to Normal.	Yes	The backend trusts fields supplied by the client and directly includes them in the SQL update query. This is a mass assignment / unauthorized property update vulnerability . A user could change record ownership or manipulate backend-controlled BMI information. The backend should only allow name, age, height, weight, and notes, while calculating bmi and category itself and taking user_id from the verified JWT.

2. Weakness Classification Table

Weakness Found	Frontend / Backend / API / Database	Security Category	Possible Risk
Negative weight accepted	Backend	Missing backend validation	Invalid data stored
User accesses another BMI record	API / Backend	Broken Object Level Authorization	Privacy violation
Normal user accesses admin route	API / Backend	Broken Function Level Authorization	Unauthorized admin action
API returns password_hash	API / Backend	Sensitive data exposure	Credential risk
XSS payload executes	Frontend	XSS	Browser script execution
SQL Injection input affects query	Backend / Database	SQL Injection	Unauthorized data access
User changes user_id or role	API / Backend	Mass assignment	Privilege or ownership manipulation

3. Problem–Solution Mapping Table

Problem	Root Cause	Where to Fix?	Proposed Solution	How to Retest?
Invalid BMI accepted	Backend does not validate	Backend	Validate name, age, height, weight	Submit invalid data again
User accesses other record	No ownership check	Backend API	Compare JWT user_id with record user_id	Access other user's record
User accesses admin route	No role check	Backend API	Check role from verified JWT	Use user token on admin route
password_has h returned	API returns all fields	Backend API	Select only safe fields	Check JSON response
XSS executes	Unsafe rendering	Frontend	Use {{ }} or textContent	Submit XSS payload again
SQL Injection possible	String SQL query	Backend	Use prepared statement	Try injection input again
user_id can be changed	Blind update	Backend API	Allow only safe fields	Send user_id in PUT request

4. Before-and-After Evidence Table

Test No	Test Case	Before fix	After fix	Evidence
1	Add BMI with invalid data	The API returned 201 Created and created a BMI record even though the data was invalid. It accepted an empty name, age -5, height 0, and weight -70.	The API returned 400 Bad Request with the message "Invalid BMI data". The backend rejected the empty name, invalid age, invalid height, and invalid weight.	Figure 1 and 9
2	Access another user's BMI record	The API returned 200 OK and displayed another user's BMI record when using a token from user 1.	The API returned 403 Forbidden with the message "Access denied". The backend now checks whether the logged-in user owns the BMI record, while staff and admin are allowed to view it.	.Figure 2 and figure 10
3	Access staff route as normal user	The API returned 200 OK and displayed all BMI/person records even though the request was made by a normal user.	The API now returns 401 Unauthorized with the message " Authentication required ". The staff route is no longer accessible without valid authentication, preventing unauthorized users from viewing BMI/person records.	Figure 3 and 11
4	Access admin route as normal user	The API returned 200 OK and exposed all user records, including sensitive fields such as passwords and password hashes, even when accessed by a normal user.	The API now returns 401 Unauthorized with the message " Authentication required ". The admin route is protected and no longer exposes user records or sensitive information to unauthorized users.	Figure 4 and 12

5	Submit XSS payload in notes	The backend accepted and stored the XSS payload in the notes field. Postman returned it as a JSON string, and no alert was executed.	The XSS payload was stored successfully but displayed as plain text, and no alert box appeared.	Figure 5 and 13
6	Check API response	The API returned user details together with sensitive fields such as password: password123 and password_hash: password123 in the response from /api/profile.	The response no longer exposed password or password_hash. Only safe user fields such as id, name, email, role, and created_at were returned.	Figure 6 and 14
7	Try SQL Injection input	The login request using aiman@student.utm.my' -- with password anything returned 200 OK and logged in successfully, even though the password was wrong.	The login API returned 401 Unauthorized with Invalid login. The SQL injection payload was treated as normal input and did not bypass the password check.	Figure 7 and 15
8	Try to update protected fields	The backend accepted and updated the protected fields user_id, bmi, and category. The record owner was changed to user ID 2, while the BMI was changed to 10 and the category to Normal.	The request containing only protected fields was rejected, and no record values were changed.	Figure 5 and 16

Figure 1:Add BMI with invalid data (before)

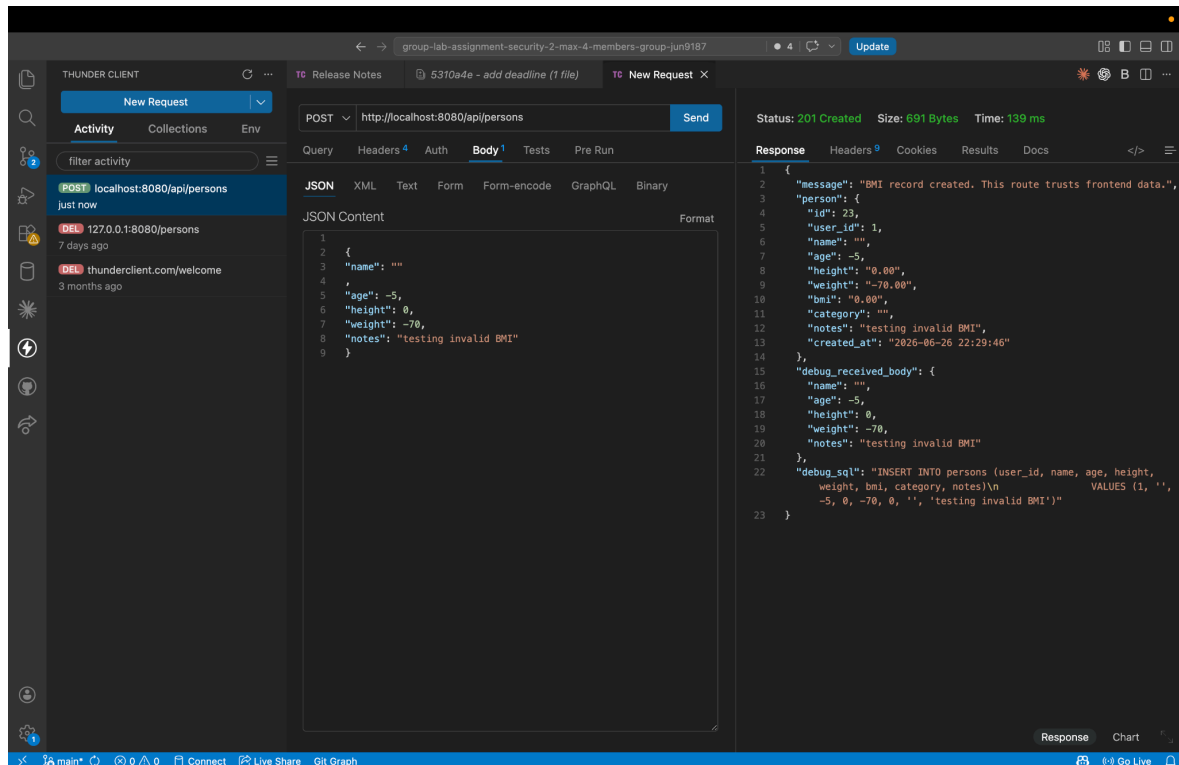


Figure 2: Access another user's BMI record (before)

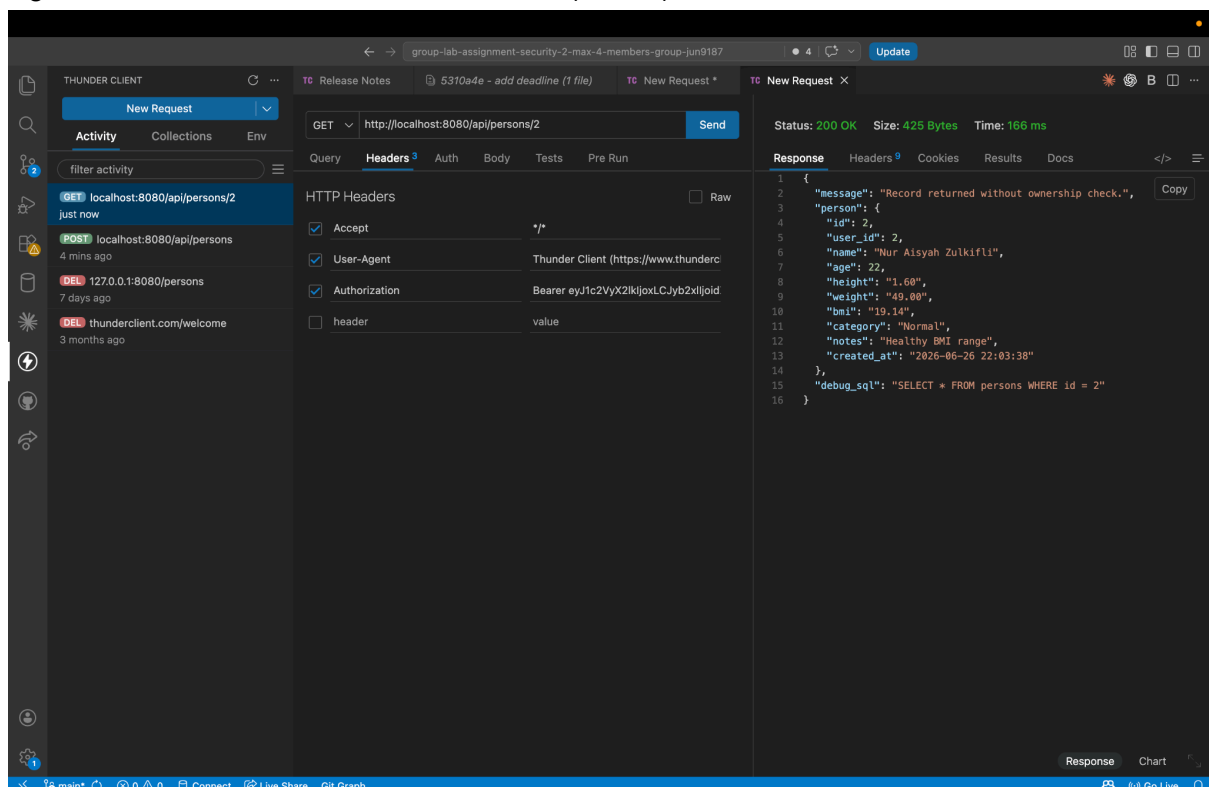


Figure 3: Access staff route as normal user(before)

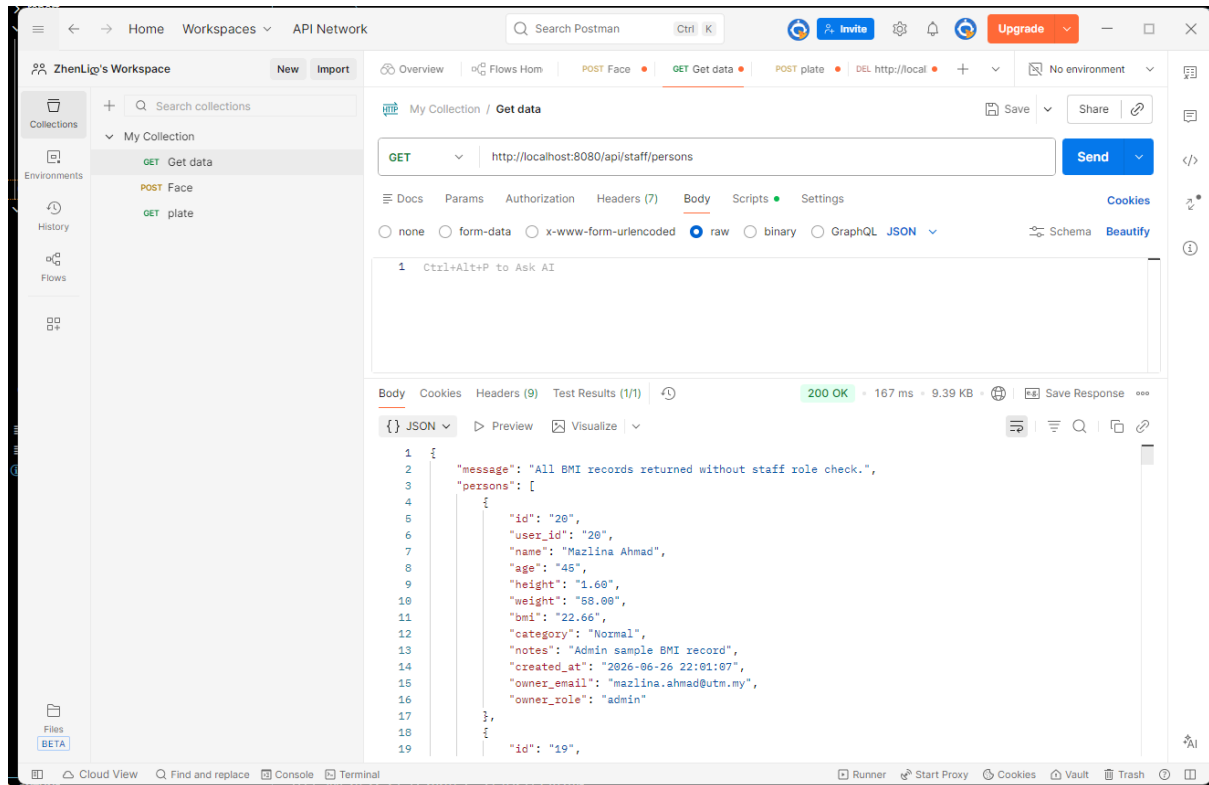


Figure 4: Access admin route as normal user(before)

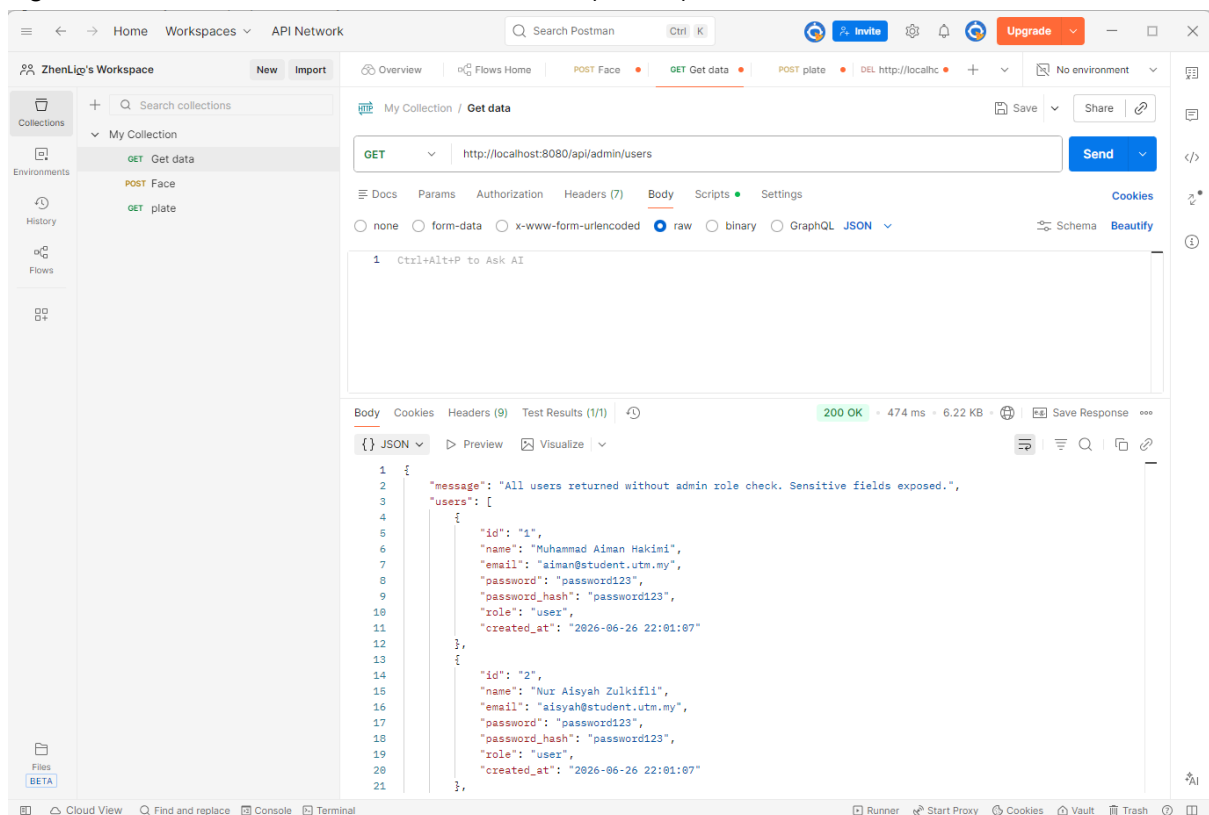


Figure 5: Submit XSS payload in notes (before)

HTTP WEBTECH_LAB / Test5

PUT http://localhost:8080/api/persons/1

Docs Params Authorization Headers (9) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "name": "Ali",
3   "age": 22,
4   "height": 1.7,
5   "weight": 65,
6   "notes": "<img src=x onerror=\"alert('XSS')\">"
7 }
```

Body Cookies Headers (9) Test Results

{ JSON Preview Visualize

```
1 {
2   "message": "BMI record updated. This route allows unsafe field updates.",
3   "person": {
4     "id": 1,
5     "user_id": 2,
6     "name": "Ali",
7     "age": 22,
8     "height": "1.70",
9     "weight": "65.00",
10    "bmi": "10.00",
11    "category": "Normal",
12    "notes": "<img src=x onerror=\"alert('XSS')\">",
13    "created_at": "2026-06-26 22:01:11"
14  },
15  "debug_received_body": {
16    "name": "Ali",
17    "age": 22,
18    "height": 1.7,
19    "weight": 65,
20    "notes": "<img src=x onerror=\"alert('XSS')\">"
21  },
22 }
```

Figure 6: Check API response (before fix)

GET ▼ <http://localhost:8080/api/profile> Send

Query Headers² Auth Body Tests Pre Run

Query Parameters

parameter	value
-----------	-------

Status: 200 OK Size: 395 Bytes Time: 99 ms Response ▼

```
1 {
2   "message": "Profile returned. This route trusts insecure token/default user.",
3   "user": {
4     "id": 1,
5     "name": "Muhammad Aiman Hakimi",
6     "email": "aiman@student.utm.my",
7     "password": "password123",
8     "password_hash": "password123",
9     "role": "user",
```

Response Chart ↕

Figure 7: Try SQL Injection (before fix)

POST ▼ <http://localhost:8080/api/login> Send

Query Headers² Auth Body¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "email": "aiman@student.utm.my' -- ",
3   "password": "anything"
4 }
```

Status: 200 OK Size: 733 Bytes Time: 129 ms Response ▼

```
1 {
2   "message": "Login successful. This token is intentionally insecure.",
3   "token": "eyJ1c2VyX2lkIjoxLCJyb2x1IjoiaXNlciIsImVtYWlsIjoiaWwltYW5Ac3R1ZGVudC51dG0ubXkiLCJub3RlIjoiaSU5STR
4   "user": {
5     "id": 1,
6     "name": "Muhammad Aiman Hakimi",
7     "email": "aiman@student.utm.my",
8     "password": "password123",
```

Copy Response Chart ↕

Figure 8: Try to update protected fields (before)

HTTP WEBTECH_LAB / Test5

PUT http://localhost:8080/api/persons/1

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "user_id": 2,
3   "bmi": 10,
4   "category": "Normal"
5 }
```

Body Cookies Headers (9) Test Results

{ } JSON Preview Visualize

```
1 {
2   "message": "BMI record updated. This route allows unsafe field updates.",
3   "person": {
4     "id": 1,
5     "user_id": 2,
6     "name": "Muhammad Aiman Hakimi",
7     "age": 21,
8     "height": "1.70",
9     "weight": "65.00",
10    "bmi": "10.00",
11    "category": "Normal",
12    "notes": "Student sample BMI record",
13    "created_at": "2026-06-26 22:01:11"
14  },
15  "debug_received_body": {
16    "user_id": 2,
17    "bmi": 10,
18    "category": "Normal"
19  },
20  "debug_sql": "UPDATE persons SET user_id = 2, bmi = 10, category = 'Normal' WHERE id = 1"
21 }
```

After Fix

Figure 9: Add BMI with invalid data (after)

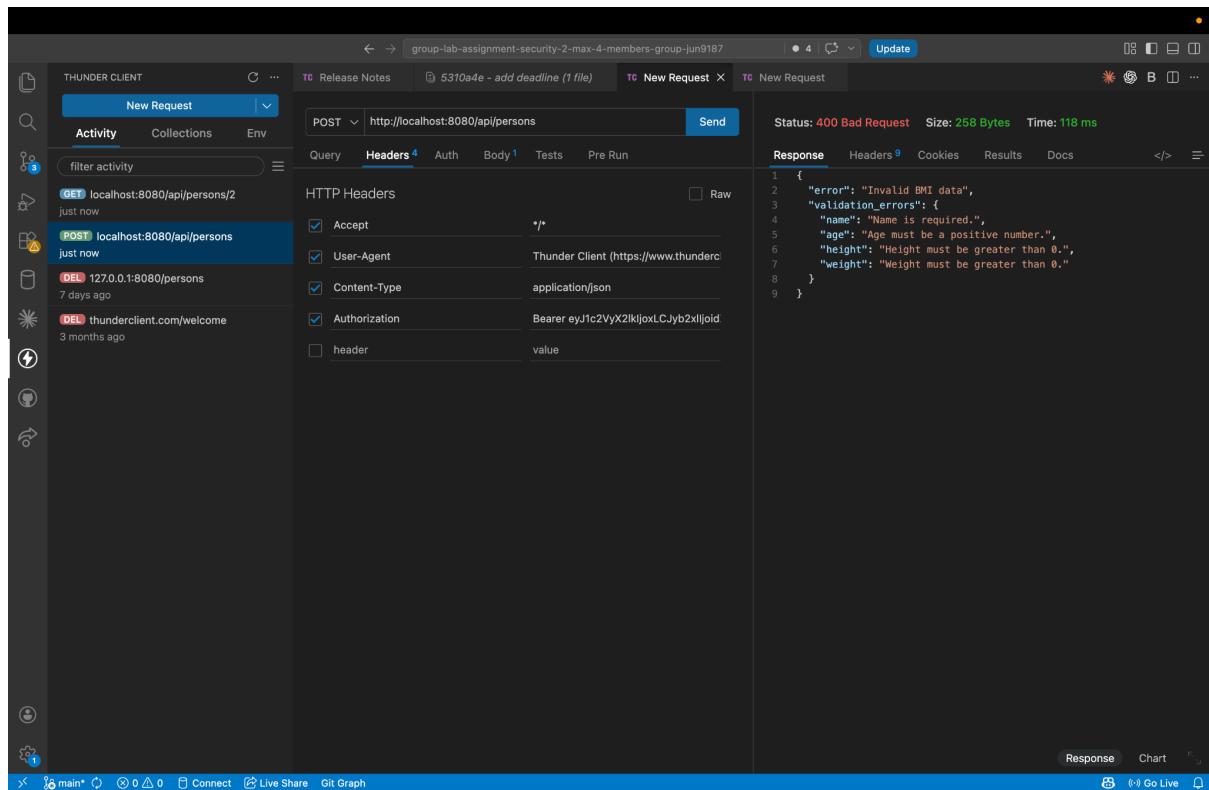


Figure 10: Access another user's BMI record (after)

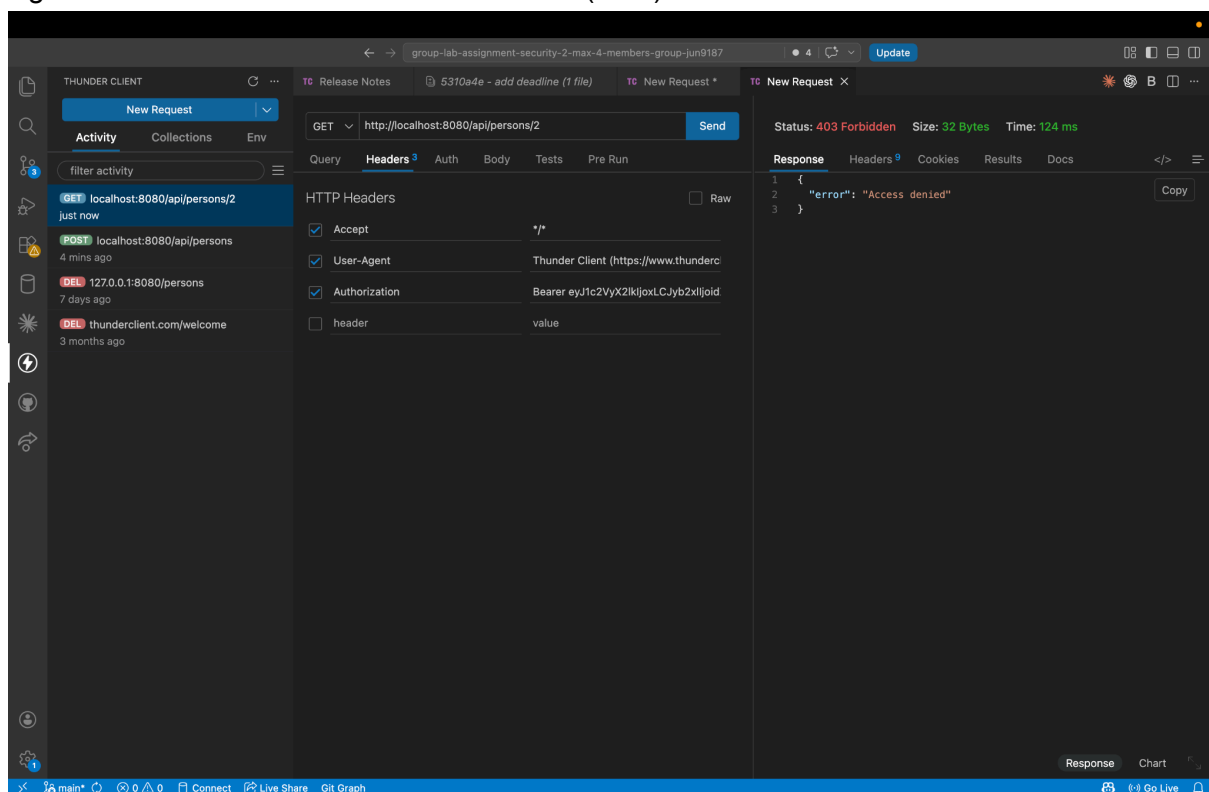


Figure 11: Access admin route as normal user(after)

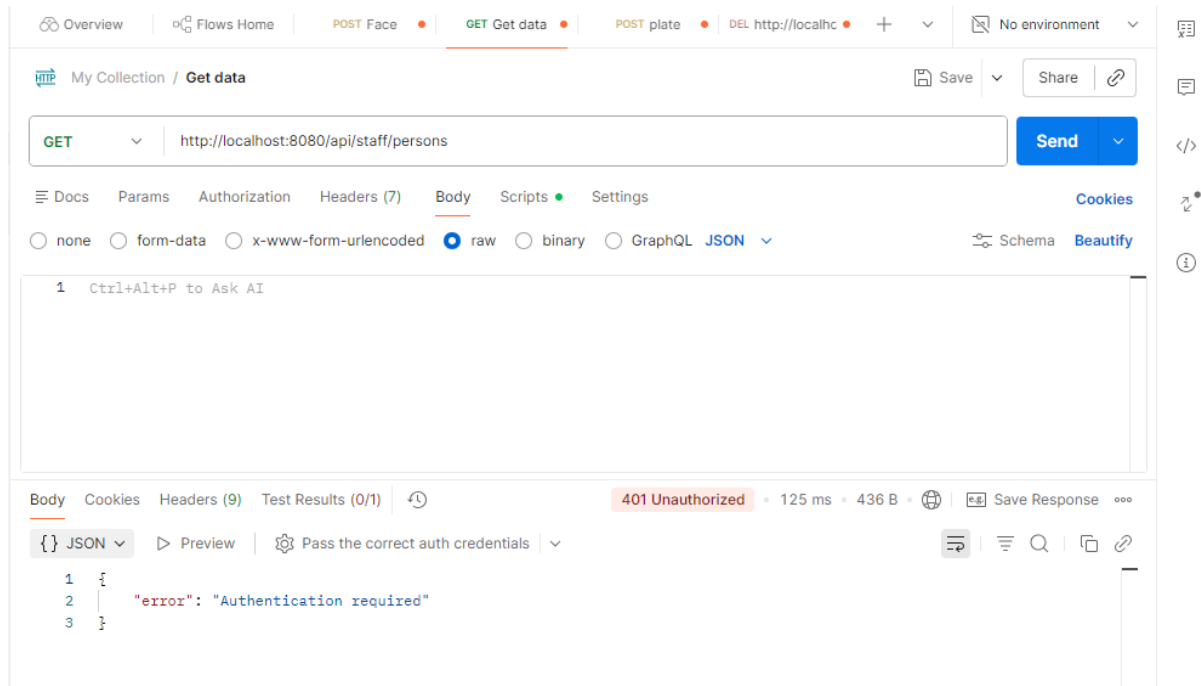


Figure 12: Submit XSS payload in notes (after)

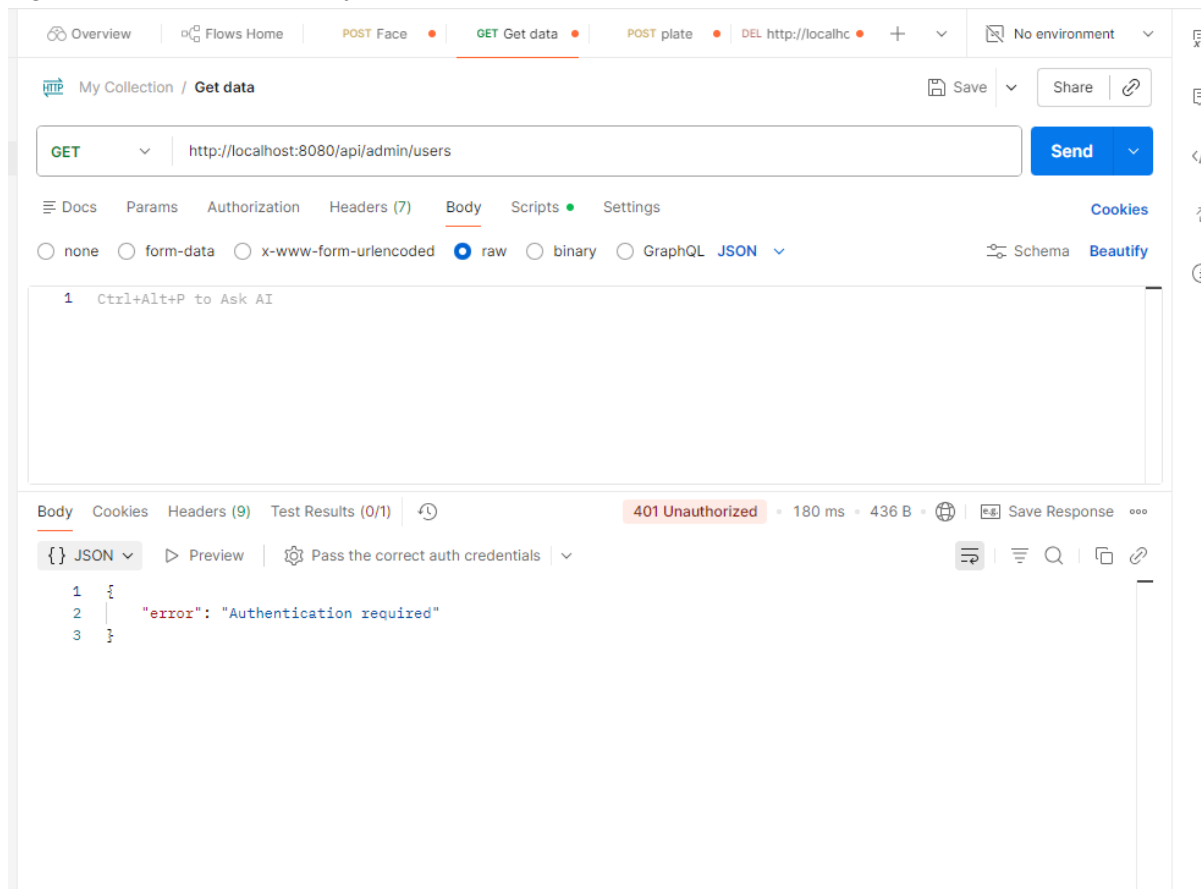


Figure 13: Submit XSS payload in notes (after)

[HTTP](#) WEBTECH_LAB / Test5

PUT ▼ http://localhost:8080/api/persons/1

☰ Docs Params Authorization Headers (9) **Body ●** Scripts Setti

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐

```
1  {
2    "name": "Ali",
3    "age": 22,
4    "height": 1.7,
5    "weight": 65,
6    "notes": "<img src=x onerror=\"alert('XSS')\">"
7  }
```

Body Cookies Headers (9) Test Results 🔄

{ } JSON ▼ ▶ Preview 🖼 Visualize ▼

```
1  {
2    "message": "BMI record updated.",
3    "person": {
4      "id": 1,
5      "user_id": 2,
6      "name": "Ali",
7      "age": 22,
8      "height": "1.70",
9      "weight": "65.00",
10     "bmi": "22.49",
11     "category": "Normal",
```

Figure 14: Check API response (after fix)

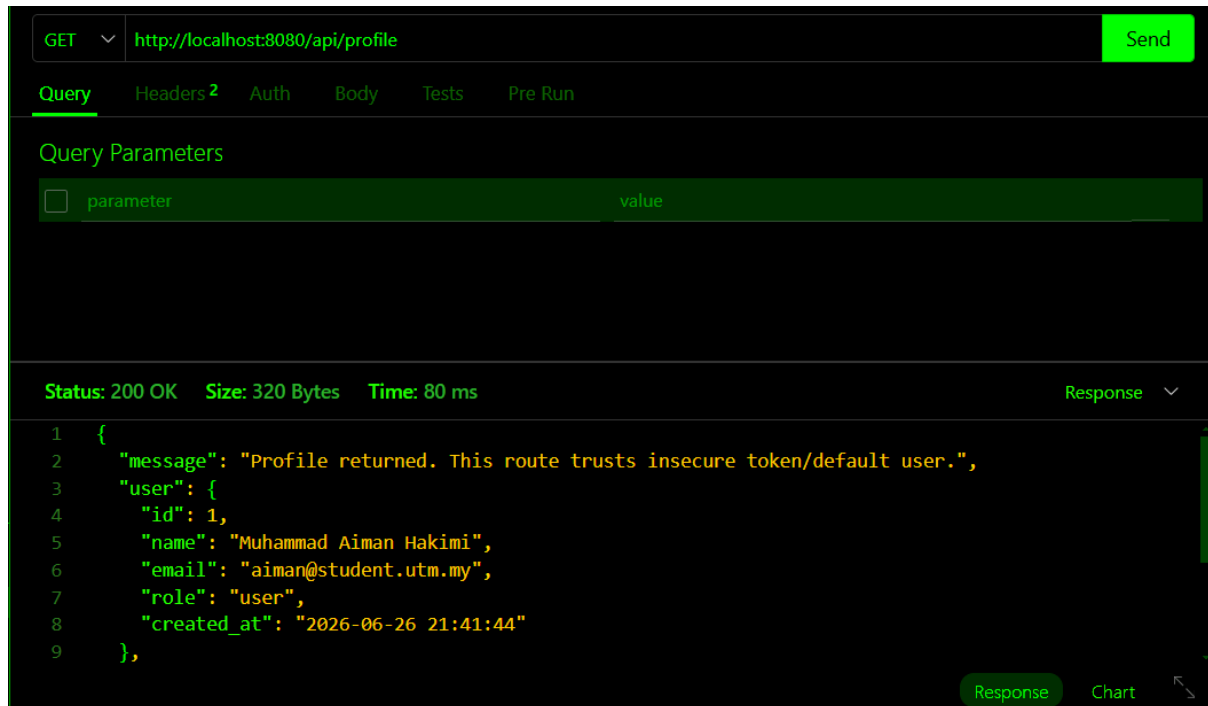


Figure 15: Try SQL Injection (after fix)



Figure 16: Try to update protected fields (after)

The screenshot displays a REST client interface with the following components:

- Request Method:** PUT
- URL:** http://localhost:8080/api/persons/1
- Request Body:**

```
{
  "user_id": 2,
  "bmi": 10,
  "category": "Normal"
}
```
- Response Body:**

```
{
  "error": "No allowed fields to update",
  "debug_received_body": {
    "user_id": 2,
    "bmi": 10,
    "category": "Normal"
  }
}
```

The response indicates that the update operation failed due to protected fields.

5. Reflection Answers

1. Which weakness was easiest to find? Why?
The easiest weakness to find was sensitive data exposure because the API response clearly showed fields such as password and password_hash in the raw JSON response.
2. Which weakness was most dangerous? Why?
The most dangerous weakness was SQL Injection because an attacker could bypass login by entering a malicious email payload and logging in without the correct password.
3. Why is frontend validation not enough?
Frontend validation is not enough because users can bypass the browser interface and send requests directly using tools such as Postman or DevTools. Security rules must be enforced by the backend.
4. Why should BMI calculation be performed at the backend?
BMI calculation should be performed at the backend because frontend values can be modified by users. The backend should calculate BMI using trusted height and weight values to prevent fake or manipulated BMI records.
5. What is the difference between authentication and authorization?
Authentication checks who the user is, such as verifying email and password.
Authorization checks what the user is allowed to access or do after logging in.
6. What is the difference between RBAC and owner-based access control?
RBAC gives access based on user roles such as user, staff, or admin. Owner-based access control gives access based on whether the data belongs to the logged-in user.
7. Why is JWT alone not enough to protect all data?
JWT only identifies the user if it is properly verified. The backend must still check ownership, roles, expiry, and permissions before allowing access to protected data.
8. Why should API responses exclude password_hash?
API responses should exclude password_hash because it is sensitive data. If exposed, attackers could try to crack the hash offline and recover the original password.
9. Why is Vue route guard not enough to protect backend routes?
Vue route guards only hide pages in the frontend. Attackers can still call backend API routes directly, so the backend must enforce access control.
10. How did you prove that your security fix works?

I repeated the same tests before and after the fix. After the fix, the API no longer returned password or password_hash, and the SQL Injection login payload returned 401 Unauthorized instead of logging in successfully.